





Advanced Relational Algebra: JOIN, Division & Querying

Connecting Relations, Discovering Patterns, Answering 'ALL' Questions



Learning Objectives

By the end of this lecture, we should be able to...

01

Explain why JOIN is needed to combine related tuples from multiple relations.

02

Distinguish Theta Join, Equi Join, Natural Join, and the main Inner/Outer Join forms.

03

Apply different JOIN operations and recognize when each form is appropriate.

04

Recognize Division queries through the “ALL required values” pattern.

05

Connect advanced relational algebra with SQL, query optimization, and efficient DBMS execution.



haribansh@bhu.ac.in

Revision of Relational Algebra

Basic operations remain the building blocks

Operation	Symbol	Purpose
Selection	σ	Select rows
Projection	π	Select columns
Union	$\cup$	Combine relations
Difference	$-$	Find differences
Cartesian Product	$\times$	Combine every tuple
Rename	ρ	Rename relation

ADVANCED

JOIN ($\bowtie$)

ADVANCED

Outer JOIN

ADVANCED

Division ($\div$)
“associated with ALL”

Think of advanced operations as combinations and patterns built on the earlier toolkit.



haribansh@bhu.ac.in

What if All data is in SINGLE table?

StudentID	StudentName	Dept	CourseID	CourseName	Teacher
101	Ram	CSE	C01	DBMS	Sharma
101	Ram	CSE	C02	OS	Verma
101	Ram	CSE	C03	Networks	Singh
102	Sita	ECE	C01	DBMS	Sharma
103	Vijay	CSE	C01	DBMS	Sharma



haribansh@bhu.ac.in

Why Do We Need JOIN?

Separate tables reduce redundancy — but create a combination problem

STUDENT

RollNo	Name	DeptID
101	Rahul	10
102	Anita	20

DeptID

DEPARTMENT

DeptID	Department
10	Computer Science
20	Electronics

Question

“Display student name with department name.”

JOIN combines the related information using DeptID.



haribansh@bhu.ac.in

JOIN Operation

Combine tuples from two relations using a related attribute or condition

General form

Relation1 ⋈ **Condition** **Relation2**

STUDENT

DeptID
10 • Rahul
20 • Anita

DEPARTMENT

DeptID
10 • CSE
20 • ECE

Condition: Student.DeptID = Department.DeptID

Name	Department
Rahul	Computer Science
Anita	Electronics



haribansh@bhu.ac.in

JOIN Classification

Four important categories to recognize



Theta $\rightarrow$ Equi is a special case. Outer JOIN has left, right, and full forms.



haribansh@bhu.ac.in

Theta Join (θ Join)

Use a comparison operator to decide which tuples combine

Allowed comparison operators



General form: $R \bowtie_{\theta} S$

Use Theta Join when the relationship is defined by a comparison.



haribansh@bhu.ac.in

Equi Join

A Theta Join where the comparison is equality (=)

STUDENT

RollNo	DeptID
101	10

DEPARTMENT

DeptID	Name
10	CSE

Student.DeptID = Department.DeptID

Result → RollNo: 101 | Name: CSE

Most common join type in practical database applications.



haribansh@bhu.ac.in

Natural Join

Automatically match same-name, compatible attributes

STUDENT

RollNo | DeptID
101 | 10

DEPARTMENT

DeptID | Department
10 | CSE

Common attribute

DeptID

RollNo	DeptID	Department
101	10	CSE

ADVANTAGE

Less writing; common attributes identified automatically

LIMITATION

Same-name unrelated attributes can cause unexpected results



Inner Join

Return only tuples with matching values in both relations

STUDENT

DeptID
10
30

DEPARTMENT

DeptID
10
20

Matching value

10

Result

DeptID = 10

Unmatched records are removed.



haribansh@bhu.ac.in

Outer Join

Keep matches — and preserve unmatched records

Sometimes the missing match is itself important information.

LEFT OUTER JOIN

Keep all left tuples

RIGHT OUTER JOIN

Keep all right tuples

FULL OUTER JOIN

Keep all tuples from both

Examples: students without departments • departments without students

Outer JOIN = matching information + preserved gaps



haribansh@bhu.ac.in

Left Outer Join

All left-side records survive

STUDENT — LEFT

Name | DeptID
Rahul | 10
Amit | 30

DEPARTMENT — RIGHT

DeptID | Department
10 | CSE

Name	Department
Rahul	CSE
Amit	NULL

Amit is retained even without a matching department.



haribansh@bhu.ac.in

Right Outer Join

All right-side records survive

STUDENT — LEFT

DeptID
10

DEPARTMENT — RIGHT

DeptID | Department
10 | CSE
20 | ECE

Department
CSE
ECE

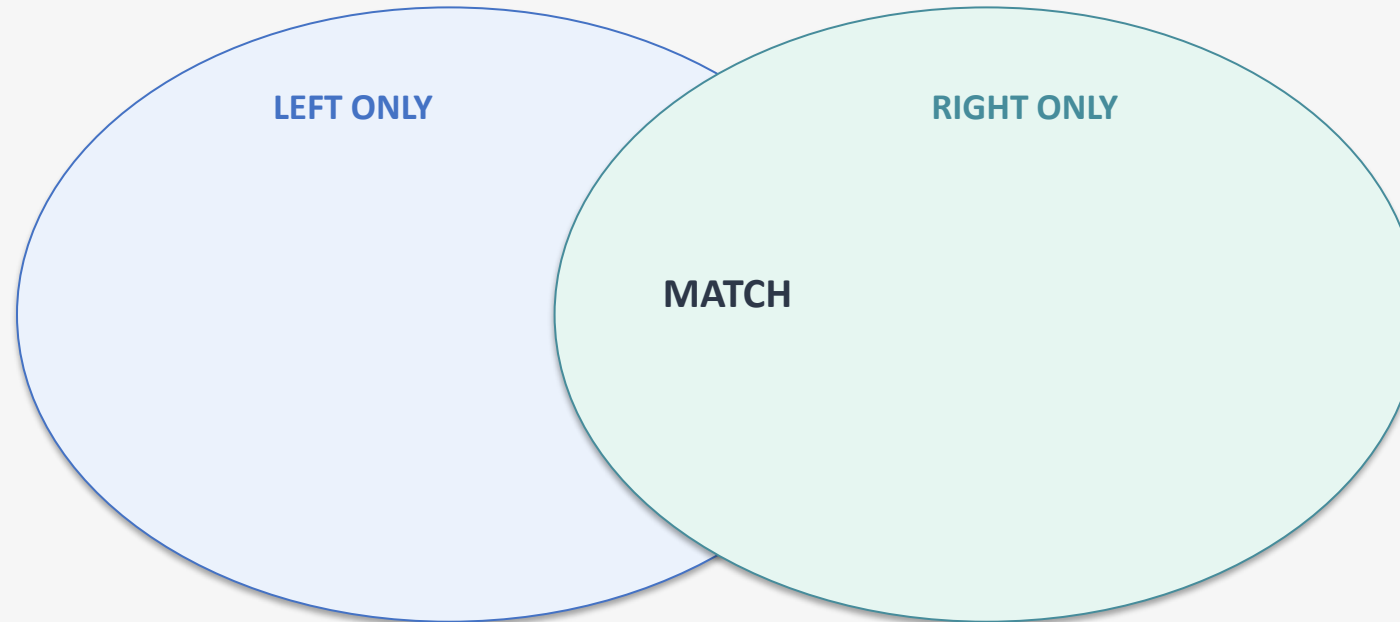
ECE appears even though no student belongs to it.



haribansh@bhu.ac.in

Full Outer Join

Preserve matching, left-only, and right-only tuples



✓ Students with departments ✓ Students without departments

✓ Departments without students



haribansh@bhu.ac.in

JOIN Examples in Real Applications

The same pattern appears across domains

BANKING

Tables
Customer
Account

Query
Find customer name and account balance.

Operation
JOIN

HOSPITAL

Tables
Patient
Appointment

Query
Find patient appointment history.

Operation
JOIN

UNIVERSITY

Tables
Student
Course
Enrollment

Query
Find students registered in DBMS.

Operation
JOIN

Different domain, same idea: connect related records through relationships.



haribansh@bhu.ac.in

Division Operation ($\div$)

The “ALL” pattern in relational algebra

Division answers questions of the form:

“Find entities associated with ALL values of another relation.”

STUDENTS

completed ALL courses

SUPPLIERS

supply ALL parts

EMPLOYEES

worked on ALL projects

Symbol: $\div$



haribansh@bhu.ac.in

Division Example

Find students who completed all required courses

Student	Course
Rahul	DBMS
Rahul	AI
Anita	DBMS

Required_Course
DBMS
AI

Query

$\text{Student_Course} \div \text{Required_Course}$

Result: Rahul

Rahul → ✓ DBMS ✓ AI

Anita → ✓ DBMS ✗ AI



haribansh@bhu.ac.in

Relational Algebra and SQL Relationship

Relational algebra provides the theoretical foundation of SQL

SELECTION

Relational Algebra
 $\sigma_{\text{Department}='CSE'}(\text{Student})$

SQL
SELECT * FROM Student
WHERE Department='CSE';

JOIN

Relational Algebra
 $\text{Student} \bowtie \text{Department}$

SQL
SELECT *
FROM Student JOIN Department
ON Student.DeptID = Department.DeptID;

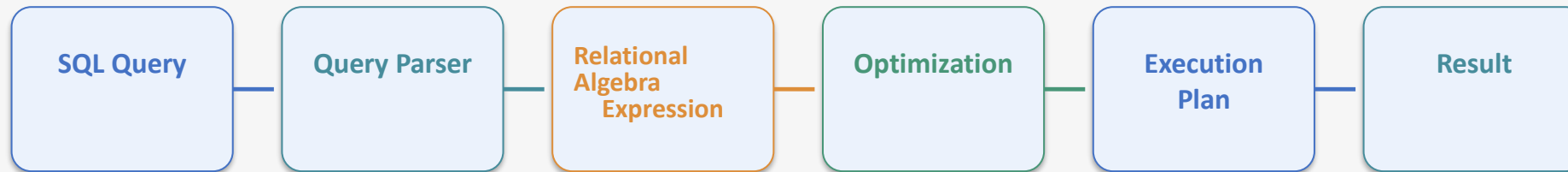
Symbols express operations; SQL expresses the query in a declarative form.



haribansh@bhu.ac.in

Role of Relational Algebra in Query Optimization

From SQL text to an efficient execution plan



Why optimize?

USE INDEXES

Reduce search work

REDUCE WORK

Avoid unnecessary operations

CHOOSE JOINS

Use efficient join methods



haribansh@bhu.ac.in

Modern Database Perspective

Different database models combine related data in different ways

RELATIONAL DATABASES

JOIN is fundamental.

Examples:

- Banking
- ERP
- Airline systems
- E-commerce

NoSQL DATABASES

Traditional joins are less common.

MongoDB uses:

- Embedded documents
- Aggregation pipeline
- References

GRAPH DATABASES

Relationships are directly represented as edges.

Example:

Person → FRIEND → Person

Important point: the need to combine and analyze related data still exists.



haribansh@bhu.ac.in

Summary

The operations and patterns to remember

Need for JOIN

Separate relations must often be combined.

JOIN

Combines related tuples using a condition.

Theta / Equi / Natural

Comparison-based, equality-based, or automatic same-name matching.

Inner / Outer

Inner keeps matches; outer joins preserve selected unmatched tuples.

Division

Recognize the “ALL required values” pattern.

SQL & Optimization

Relational algebra underpins query reasoning and optimization.



haribansh@bhu.ac.in

Key distinction: Selection → rows | Projection → columns | JOIN → relations

Learning Reflections

Explain the idea in your own words

- 1 What is the difference between Inner Join and Outer Join? Give one example where preserving unmatched records matters.
- 2 How is Equi Join different from Natural Join?
- 3 Why is the word “ALL” a useful clue for recognizing a Division query?
- 4 Why does a DBMS translate and optimize query operations before execution?



haribansh@bhu.ac.in

Homework

Explain how advanced relational algebra operations extend the basic querying capabilities of relational databases, and describe how Theta Join, Equi Join, Natural Join, Inner Join, Outer Join, and Division are used to combine relations and answer complex queries. Explain the purpose, condition or matching rule, input and output of each operation, and illustrate how JOIN and Division relate to SQL queries and real-world database applications.



haribansh@bhu.ac.in



Thanks!



📌 Heads Up!

These slides are just a **guide** — not the full story!

To really learn , make sure to read your book, and explore other resources too.

haribansh@bhu.ac.in